

Ορλάνδος Αλέξιος sdi2300149

Για την υλοποίηση της εργασίας, στον αρχικό σκελετό του xv6-riscv-riscv repository που δόθηκε έγιναν αλλαγές στους φακέλους kernel user, και στο makefile.

1. Η εντολή ps

Για την υλοποίηση αυτής της εντολής πραγματοποιήθηκαν οι εξής αλλαγές:

1. **kernel/pstat.h:**
 - Νέο αρχείο: struct pstat, που περιέχει τα απαραίτητα στοιχεία για την getpinfo() (pid,ppid,name,priority,state,size)
2. **kernel/proc.h:**
 - struct proc{}: Ορισμός νέων μεταβλητών στο struct proc. Περιέχουν το priority του process, πόσα ticks έχει έχει υπάρξει ενεργό, και πόσα ticks περιμένει.
3. **kernel/proc.c:**
 - allocproc(void): Αρχικοποίηση των νέων μεταβλητών που προστέθηκαν στο struct proc για το νέο process που δημιουργείται.
 - kfork(void): Αρχικοποίηση των νέων μεταβλητών που προστέθηκαν στο struct proc για το νέο process που δημιουργείται από το fork. Ως προς την προτεραιότητα δεν υπάρχει κληρονομικότητα. Οι διεργασίες όλες ξεκινάνε παρόμοια.
4. **kernel/syscall.h:**
 - define SYS_getpinfo 22: Ορισμός id για το νέο syscall.
5. **kernel/syscall.c:**
 - extern uint64 sys_getpinfo(void):
 - [SYS_getpinfo] sys_getpinfo: Απαραίτητες δηλώσεις για την νέα συνάρτηση
6. **kernel/sysproc.c:**
 - sys_getpinfo: Διασχίζει τα processes, αντιγράφοντας τις πληροφορίες τους σε ένα struct pstat, το οποίο περνάει στον user με το copyout().
7. **user/user.h:**
 - int getpinfo(struct pstat*): Απαραίτητες δηλώσεις για την νέα συνάρτηση.
8. **user/usys.pl:**
 - entry("getpinfo"): Απαραίτητες δηλώσεις για την νέα συνάρτηση.
9. **user/ps.c:**
 - main(): Υλοποίηση της εντολής ps. Χρησιμοποιεί της getpinfo() και εκτυπώνει τα αποτελέσματα.
10. **user/hog.c:**
 - main(): Χρησιμοποιείται για extra έλεγχο του ps. Τρέξε το παρακάτω (hog & hog & hog &), και περιοδικά τρέξε και την ps. Παρατηρούμε σωστή εκτύπωση. Έχοντας υλοποιήσει και το MLFQ scheduler παρατηρούμε αλλαγή priority με το πέρασμα του χρόνου.

2. MLFQ scheduler

Με σκοπό την σωστή ενημέρωση των μεταβλητών priority στο struct proc κατά την διάρκεια της εκτέλεσης και την εκμετάλλευση αυτών με σκοπό την εφαρμογή MLFQ scheduler, πραγματοποιήθηκαν οι εξής αλλαγές:

1. kernel/proc.c:

- update_time: Νέα συνάρτηση η οποία καλείται σε κάθε timer tick από την CPU. Ενημερώνει τα ticks των διεργασιών και υλοποιεί τους κανόνες υποβιβασμού (demotion) όταν εξαντλείται το χρονομερίδιο (4, 8, 16, 32 ticks), καθώς και τον κανόνα αναβάθμισης (promotion) για την αποφυγή του starvation.
- higher_priority_exists(): Βοηθητική συνάρτηση που ελέγχει αν υπάρχει διαθέσιμη διεργασία με υψηλότερη προτεραιότητα από την τρέχουσα..
- scheduler: Πλήρης ανασχεδιασμός του scheduler ώστε να υποστηρίζει 4 επίπεδα προτεραιότητας. Χρησιμοποιείται ο στατικός πίνακας last_p[4] για την εφαρμογή του Round Robin εντός κάθε επιπέδου, διασφαλίζοντας ότι η αναζήτηση ξεκινά από την επόμενη διεργασία του πίνακα proc. Μόλις μια διεργασία εκτελεστεί, ο έλεγχος επανέρχεται αμέσως στην υψηλότερη προτεραιότητα.

2. kernel/trap.c

- clockintr(): Προσθήκη κλήσης της update_time() υπό την συνθήκη cpuid() == 0, ώστε να αποφευχθεί η πολλαπλή ενημέρωση των ticks σε συστήματα με περισσότερους από έναν πυρήνες.
- usertrap() // kerneltrap(): Τροποποίηση της λογικής παραχώρησης της CPU (yield). Η διεργασία πλέον δεν εγκαταλείπει τη CPU αυτόματα σε κάθε timer tick. Η κλήση της yield() γίνεται πλέον επιλεκτικά, μόνο στην περίπτωση που η διεργασία εξαντλήσει το χρονομερίδιο που αντιστοιχεί στο επίπεδο προτεραιότητας της ή αν η higher_priority_exists() επιστρέψει αληθές. Αυτό επιτρέπει στις διεργασίες χαμηλής προτεραιότητας να εκτελούνται για μεγαλύτερα συνεχή διαστήματα εφόσον δεν υπάρχει ανταγωνισμός από υψηλότερες προτεραιότητες.

3. kernel/defs.h:

- void update_time(void): :
- void higher_priority_exists(void): : Απαραίτητες δηλώσεις.

4. /makefile:

- \$U/ ps\:
- \$U/ hog\: Απαραίτητες δηλώσεις για σωστή λειτουργία makefile.